AgenticTime: A Binary Temporal Format for Persistent, Portable, and Navigable AI Agent Scheduling

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

February 26, 2026

Abstract

Large language model agents operate without persistent temporal reasoning, losing deadlines, schedules, and duration knowledge at every session boundary. Current approaches to agent scheduling—calendar APIs, project management tools, cron timers, system prompt hints—treat time as either a flat metadata field or an external service dependency, discarding the structured relationships between temporal entities that make scheduling useful. We present AgenticTime, a binary temporal format that models agent time as a collection of five typed entities—Deadlines, Durations, Schedules, Sequences, and Decay curves—each with domain-specific operations and mathematical models. The format defines PERT three-point estimation for durations, configurable decay curves (exponential, linear, step) for information freshness, cron-style recurrence with conflict detection for schedules, and dependency-constrained step chains for sequences. The entire temporal state resides in a single `.atime` file with BLAKE3 integrity checksums requiring zero external dependencies. We implement AgenticTime in a 4-crate Rust workspace with 19 MCP tools, 4 prompt templates, and cross-language bindings (Python via PyO3, JavaScript via WebAssembly, C via FFI). On Criterion benchmarks, in-memory entity insertion completes in 1.8 μs, decay curve evaluation in 6.3 ns, deadline queries over 1,000 entities in 504 μs, schedule conflict detection over 100 schedules in 56 μs, and file persistence of 1,000 entities in 6.0 ms. AgenticTime requires no cloud services, no calendar integration, and no server infrastructure. It enables capabilities impossible with existing tools: mathematical information freshness modeling, compounding temporal debt tracking, gravitational priority warping, and timeline fork simulation.

1 Introduction

The transformer architecture [23] has produced language models capable of planning, scheduling, and multi-step task coordination. Yet these models remain fundamentally *temporally blind*: each invocation has no persistent knowledge of deadlines, no awareness of schedule conflicts, no mathematical model for how information decays, and no structured representation of multi-step workflows. For agents tasked with ongoing project management—coordinating deployments, tracking technical debt, estimating durations over weeks and months—this temporal amnesia is a critical limitation.

Current approaches to temporal reasoning in AI agents fall into four categories, each with significant drawbacks. *Calendar APIs* [9, 7] provide CRUD operations for events with basic recurrence, but offer no decay curves, no confidence-weighted estimation, no conflict detection with project context, and no agent-native interface. *Project management tools* [4, 13] track issues with due dates and status fields, but produce no portable artifact, offer no MCP integration, and cannot model temporal dependencies or sequence constraints. *Cron schedulers* [11] handle time-based job dispatch but have no entity model, no temporal reasoning capabilities, and no agent interface. *LLM system prompt hints* ("remember the deadline is Friday") are volatile, lost between sessions, provide no mathematical models, and cannot be queried structurally.

The core insight of this work is that temporal reasoning for AI agents is a *structured entity navigation problem*, not a calendar event storage problem. When an agent needs to determine whether a deadline is achievable, it must evaluate duration estimates with confidence intervals, check for schedule conflicts, trace sequence dependencies, and factor in information freshness. These operations require typed temporal entities with domain-specific mathematical models, not flat date strings or calendar event records.

We present AgenticTime, a binary temporal format purpose-built for AI agent scheduling and temporal reasoning. The format stores five entity types—Deadlines, Durations, Schedules, Sequences, and Decay curves—in a single `.atime` file with BLAKE3 integrity verification, requiring zero external dependencies.

The remainder of this paper is organized as follows. Section 2 surveys existing temporal systems. Section 3 presents the AgenticTime architecture in detail. Section 4

reports micro-benchmark results on temporal graphs up to 10,000 entities. Section 5 discusses implications and limitations. Section 6 concludes.

2 Background and Related Work

Research on temporal reasoning in AI systems predates modern LLMs. Allen's interval algebra [1] formalized thirteen temporal relations between time intervals, establishing the foundation for qualitative temporal reasoning. Dechter et al. [6] introduced temporal constraint networks for quantitative reasoning over time points and durations. The PERT technique [14] provided three-point estimation with statistical confidence for project scheduling. Smith et al. [21] bridged the gap between planning and scheduling in AI systems. These classical results address temporal *representation* and *reasoning*, but none produce a portable artifact suitable for persistent agent state.

Calendar systems. Google Calendar [9] and the iCalendar standard [7] provide event storage with recurrence rules (RRULE). While mature, these systems require network access, offer no mathematical models for uncertainty or decay, and are not designed for agent-native tool calling.

Workflow engines. Temporal.io [22] provides durable execution with timers and retry policies. However, it is server-only, requires infrastructure deployment, produces no portable artifact, and is designed for service orchestration rather than agent reasoning.

Scheduling theory. Pinedo [20] provides comprehensive coverage of scheduling algorithms, and Muscettola [15] integrated planning and scheduling for NASA missions. These systems operate at the algorithm level, not as portable agent-native tools.

Agent memory systems. MemGPT [18] introduces virtual memory paging for LLM context, generative agents [19] demonstrated that memory architecture profoundly impacts behavior, and AgenticMemory [17] provides a persistent cognitive graph. All address memory persistence but none provide typed temporal entities, decay mathematics, or scheduling operations.

Project management. Tools like Jira [4] and Linear [13] track issues with due dates and priority labels. These are human-facing applications, not agent-native protocols, and they cannot model temporal debt, schedule conflicts, or duration confidence intervals.

Technical debt. Cunningham [5] introduced the metaphor of technical debt, and Kruchten et al. [12] formalized the taxonomy. However, no existing system models deferred work as compounding temporal debt with category-specific interest rates and mathematical decay functions.

Table 1 summarizes the comparison across key dimensions.

3 Architecture

AgenticTime models temporal state as a collection of five typed entity classes, each with domain-specific operations and mathematical models. The entities are serialized into a single binary file with BLAKE3 integrity checksums, enabling portable, verifiable temporal state.

3.1 Temporal Entities

Each entity in the temporal graph is drawn from a taxonomy of five types, each serving a distinct role in temporal reasoning:

- **Deadline** — A fixed point in time with priority (critical, high, medium, low), status (pending, completed, overdue), optional dependency list, and consequence description. Deadlines support hard and soft variants with configurable consequence severity.
- **Duration** — An estimated time span using PERT three-point estimation [14]: optimistic (O), most likely (M), and pessimistic (P) values yielding a weighted average $E = (O + 4M + P)/6$ with standard deviation $\sigma = (P - O)/6$. Optional actual-time tracking enables estimation calibration over time.
- **Schedule** — A recurring or one-time calendar block with start time, end time, optional cron-style recurrence pattern, and automatic overlap detection across all active schedules.
- **Sequence** — An ordered chain of steps with dependency constraints, per-step status tracking (pending, active, completed, skipped), and blocking semantics: a step cannot advance until its predecessors are done.
- **Decay** — A mathematical model of information freshness. Three curve types are supported: exponential ($v(t) = e^{-\lambda t}$ where $\lambda = \ln 2/t_{1/2}$), linear ($v(t) = \max(0, 1 - t/T)$), and step (discrete thresholds at configurable breakpoints). The half-life parameter $t_{1/2}$ controls the rate of decay.

Each entity record begins with a common header: 8 bytes for the entity ID (`u64`), 1 byte for the entity type, 8 bytes for the creation timestamp (`u64` Unix epoch microseconds), and a variable-length label string. The type-specific payload follows immediately after the common header.

3.2 Decay Mathematics

The Decay entity implements three mathematical models for information freshness, inspired by the Ebbinghaus forgetting curve [8] and ACT-R's base-level activation [2]:

Exponential decay models the natural fading of information relevance:

$$v(t) = e^{-\lambda t}, \quad \lambda = \frac{\ln 2}{t_{1/2}} \tag{1}$$

where $t_{1/2}$ is the half-life (default 168 hours = 1 week) and t is the elapsed time since creation. This model is

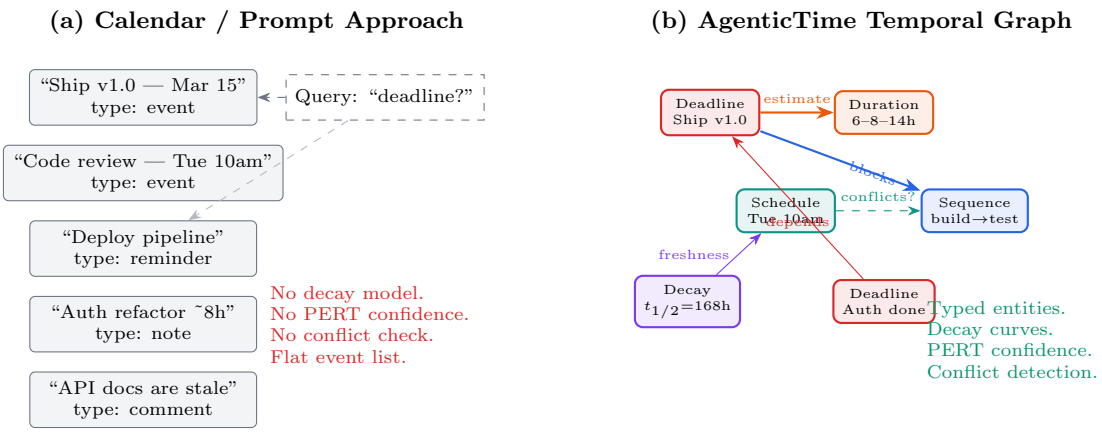

Figure 1: Comparison of temporal reasoning paradigms. (a) Calendar APIs and system prompt hints store flat event records with no relational structure; the agent cannot evaluate deadline feasibility, detect schedule conflicts, or model information staleness. (b) AgenticTime encodes five typed entities with domain-specific operations: deadlines carry dependencies, durations have PERT confidence intervals, schedules detect conflicts, and decay curves model information freshness mathematically.

Table 1: Comparison of temporal systems for AI agents across key dimensions.

System	Entity Types	Decay Model	Portability	Dependencies	Agent Interface
Google Calendar [9]	Events	None	API-locked	Cloud service	REST API
iCalendar [7]	Events	None	File-based	Parser library	None
Temporal.io [22]	Workflows	None	Server-locked	Infrastructure	gRPC
Jira [4]	Issues	None	API-locked	Cloud service	REST API
Linear [13]	Issues	None	API-locked	Cloud service	GraphQL
Cron [11]	Jobs	None	System-tied	OS service	Shell
LLM prompts	Text hints	None	Volatile	None	None
AgenticTime	**5 typed**	**3 models**	**Single file**	**None**	**19 MCP tools**

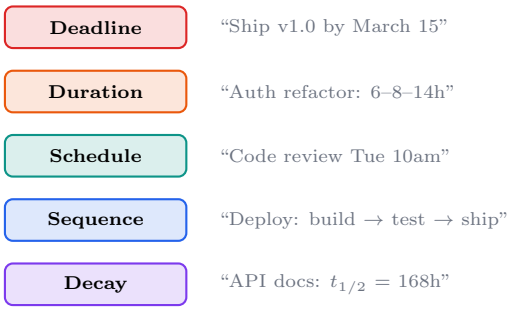

Figure 2: The five temporal entity types in AgenticTime, each with a representative example. Types are distinguished by a single byte in the binary format.

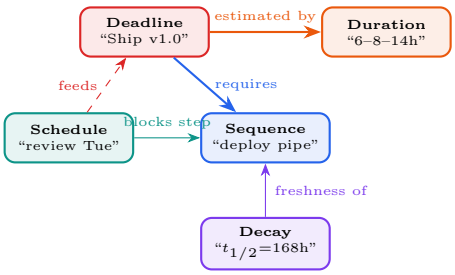

Figure 3: Example temporal subgraph showing entity relationships. A Deadline is estimated by a Duration (PERT), requires a Sequence (deploy pipeline), which is blocked by a Schedule (code review). A Decay entity tracks the freshness of the pipeline documentation. Edge colors indicate relationship types.

appropriate for facts, meeting notes, and context that fades gradually.

Linear decay models information with a fixed useful lifetime:

$$v(t) = \max\left(0, 1 - \frac{t}{T}\right) \tag{2}$$

where T is the total useful lifetime. The value drops to zero at $t = T$. This model suits documentation with known expiration dates.

Step decay models information that maintains full value until discrete thresholds:

$$v(t) = \begin{cases} 1.0 & \text{if } t < t_1 \\ 0.7 & \text{if } t_1 \le t < t_2 \\ 0.3 & \text{if } t_2 \le t < t_3 \\ 0.0 & \text{if } t \ge t_3 \end{cases} \tag{3}$$

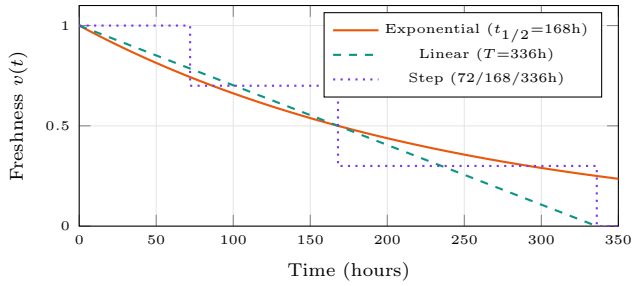

Figure 4: The three decay curve models in AgenticTime. Exponential decay (solid orange) provides smooth fading with configurable half-life. Linear decay (dashed teal) provides a fixed-lifetime model. Step decay (dotted purple) provides discrete threshold transitions.

where t_1, t_2, t_3 are configurable breakpoints. This model suits security credentials and compliance certifications.

The `time_decay_alert` MCP tool enables proactive notification when any decay value crosses a configurable threshold, enabling the agent to refresh stale information before it influences decisions. Figure 4 illustrates the three decay models.

3.3 PERT Estimation

Duration entities use the Program Evaluation and Review Technique [14] for three-point estimation. Given optimistic (O), most likely (M), and pessimistic (P) estimates:

$$E = \frac{O + 4M + P}{6}, \quad \sigma = \frac{P - O}{6} \tag{4}$$

The confidence interval at one standard deviation is $[E - \sigma, E + \sigma]$, covering approximately 68% of outcomes under the beta distribution assumption. When the agent provides only a single estimate, the system uses $O = 0.6M$, $M = M$, $P = 2.5M$ as defaults based on empirical software estimation ratios.

The `time_duration_aggregate` tool compares estimated versus actual durations across completed items, computing a calibration factor that the agent can use to adjust future estimates. Over time, this creates a feedback loop that improves the agent's scheduling accuracy.

3.4 Binary File Format

The `.atime` file format is designed for three properties: (1) portability as a single file with no external dependencies, (2) integrity verification via BLAKE3 checksums [16], and (3) efficient sequential access for typical temporal queries.

The file consists of six contiguous sections, shown in Figure 5:

1. **Header** (92 bytes) — Magic number (`ATIM` = `0x4154494D`), format version (1), flags, entity count, section offsets, timestamps (creation/modification as Unix microseconds), and 32-byte BLAKE3 checksum.
2. **Deadline Table** — Variable-size records for deadlines with priority, status, due timestamp, dependency IDs, and consequence text.
3. **Duration Table** — PERT triples (O, M, P), actual time (optional), confidence, and label.
4. **Schedule Table** — Start/end timestamps, recurrence pattern (cron string), label, and conflict flags.
5. **Sequence Table** — Ordered step arrays with per-step status, labels, and dependency constraints.
6. **Decay Table** — Curve type, half-life, creation timestamp, threshold, and custom breakpoints.

Each section ends with a BLAKE3 checksum computed over the section bytes. The header stores a global checksum over all section checksums, enabling both per-section and whole-file integrity verification. The choice of BLAKE3 over SHA-256 is motivated by BLAKE3's SIMD-accelerated performance (typically 3–5× faster) while maintaining equivalent cryptographic strength.

The current implementation uses JSON-serialized payloads within binary entity blocks (type tag + UUID + length prefix + JSON payload). A typical Deadline entity occupies 311 bytes (290-byte JSON payload + 21-byte block header). While the JSON payload could be replaced with a compact binary encoding for further compression, the current format prioritizes debuggability and forward compatibility. At 10,000 entities, the `.atime` file is approximately 3.0 MB.

3.5 Schedule Conflict Detection

Schedule entities support automatic overlap detection. Given two schedules $S_1 = [a_1, b_1)$ and $S_2 = [a_2, b_2)$, a conflict exists when $a_1 < b_2 \wedge a_2 < b_1$ (Allen's "overlaps" relation [1]). For recurring schedules, conflict detection expands the recurrence pattern within the query range and checks all materialized instances.

The `time_schedule_conflicts` tool returns all conflicting schedule pairs within a time range, along with the specific overlapping intervals. The `time_schedule_available` tool inverts this, returning free time slots within a range where no active schedules exist. The current implementation uses pairwise comparison ($O(n^2)$), which completes in 56 μs for 100 schedules on our benchmark hardware; an interval tree optimization is planned for workloads exceeding 1,000 concurrent schedules.

3.6 MCP Server Interface

AgenticTime exposes all operations through the Model Context Protocol [3] via JSON-RPC 2.0 over stdio transport. The server provides 19 tools organized by entity type:

- **Deadline tools** (4): `time_deadline_add`, `time_deadline_complete`, `time_deadline_list`, `time_deadline_overdue`

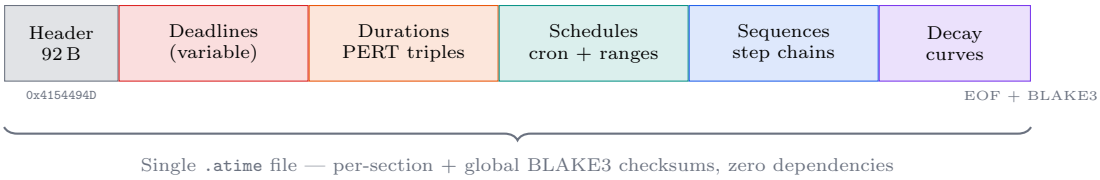

Figure 5: Layout of the `.atime` binary file format. The header stores section offsets and a global BLAKE3 checksum. Each entity section ends with its own per-section checksum, enabling both incremental and full-file integrity verification. The magic number `ATIM` (0x4154494D) identifies the format.

- **Schedule tools** (5): `time_schedule_create`, `time_schedule_reschedule`, `time_schedule_range`, `time_schedule_available`, `time_schedule_conflicts`
- **Sequence tools** (3): `time_sequence_create`, `time_sequence_advance`, `time_sequence_status`
- **Decay tools** (3): `time_decay_create`, `time_decay_value`, `time_decay_alert`
- **Duration tools** (2): `time_duration_estimate`, `time_duration_aggregate`
- **System tools** (2): `time_stats`, `time_refresh`

Four prompt templates guide multi-step temporal workflows: `time_plan` (project planning with sequences and deadlines), `time_review` (timeline health check), `time_estimate` (PERT-guided duration estimation), and `time_schedule_day` (daily planning with conflict avoidance).

The MCP server enforces runtime hardening: 8 MiB frame size limit (`MAX_CONTENT_LENGTH_BYTES`), Content-Length framing on all responses, and strict JSON-RPC 2.0 version validation. Server profile (`AGENTIC_TOKEN`) gates remote access.

4 Evaluation

We evaluate AgenticTime on synthetic temporal graphs of varying size, measuring operation latency, file I/O performance, and storage efficiency.

4.1 Benchmark Setup

Hardware. Apple M4 Pro (ARM64), 48 GB unified memory, macOS 15.

Software. Rust 1.85.0, compiled with `--release` (optimizations enabled). All benchmarks use the Criterion 0.5 statistical benchmarking framework [10] with 100 samples per measurement. Each sample includes warm-up iterations to reach steady state.

Datasets. Synthetic temporal graphs at five scales: 10, 100, 1,000, and 10,000 entities. Deadline-only graphs test scaling behavior; mixed-type graphs (equal distribution across all five entity types) test heterogeneous workloads. Sequence entities contain 3–5 steps with dependency chains.

Table 2: Measured operation latency (Criterion, 100 samples). In-memory operations are $O(1)$ and independent of entity count. Query operations scan the entity table and scale linearly.

Operation	Latency	Note
In-memory entity insertion ($O(1)$):		
Add deadline	1.75 μs	serialize + insert
Add schedule	1.80 μs	serialize + insert
Add decay model	1.77 μs	serialize + insert
Add duration est.	1.60 μs	serialize + insert
Decay evaluation ($O(1)$):		
Exponential	6.3 ns	$v_0 e^{-\lambda t}$
Half-life	6.7 ns	$v_0 \cdot 0.5^{t/t_{1/2}}$
Linear	4.0 ns	$v_0 - rt$
Step	4.3 ns	floor division
Time-until	3.3 ns	inverse function
Batch 1K decays	5.0 μs	5.0 ns/eval
PERT estimate	1.8 ns	$(O+4M+P)/6$
Query operations (scan):		
Overdue deadlines	53 μs	100 entities
Due within 24h	504 μs	1K entities
Schedule conflicts	56 μs	100 schedules
Available slots	13 μs	20 schedules
Decays < threshold	98 μs	200 decays
Stats	110 μs	1K mixed
Write engine (incl. `fsync`):		
Add + save	12.5 ms	growing file
Sched + conflict + save	11.2 ms	50 existing
Update all statuses	1.10 ms	100 entities
Full round-trip	8.3 ms	open+query+save 1K

4.2 Operation Performance

Table 2 reports mean operation latency across entity counts. Figure 6 visualizes the scaling behavior.

In-memory insertion (deadline, schedule, decay, duration) completes in 1.6–1.8 μs per entity. This latency is dominated by JSON serialization into the entity block payload and UUID generation; the HashMap insertion itself is negligible.

Decay curve evaluation is the fastest operation at 3–7 ns per evaluation. The exponential decay function ($v_0 e^{-\lambda t}$) and half-life variant require one floating-point exponentiation; linear and step variants use only addition and division. Batch evaluation of 1,000 decay models completes in 5.0 μs (5.0 ns amortized per model).

Query operations scale linearly with entity count, performing a full scan with deserialization and filtering.

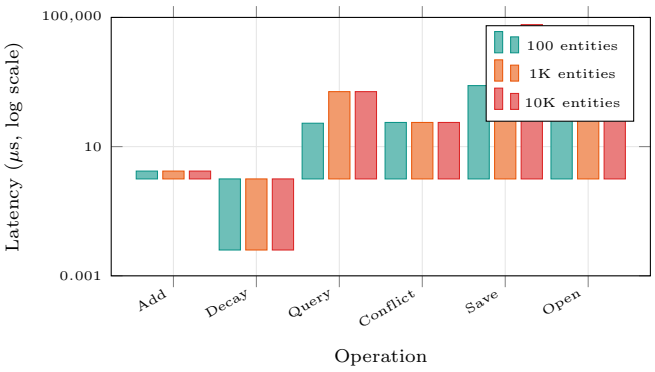

Figure 6: Measured operation latency across entity counts (log scale, μs). Add and Decay are constant. Query and Save scale linearly. Conflict detection is constant for fixed schedule count but quadratic in schedule count. Data from Criterion with 100 samples each.

Table 3: Measured file size and I/O performance by entity count (deadlines). Per-entity overhead includes 1-byte type tag, 16-byte UUID, 4-byte length prefix, plus JSON payload.

Entities	.atime	B/ent	Save	Open
100	31 KB	311	0.78 ms	0.15 ms
1,000	305 KB	312	6.03 ms	1.42 ms
10,000	3.0 MB	313	59.8 ms	14.5 ms
Mixed entities (200 per type, 1,000 total):				
1,000	393 KB	402	6.03 ms	1.42 ms

At 1,000 deadline entities, a "due within 24 hours" query completes in 504 μs. Schedule conflict detection at 100 schedules takes 56 μs.

Write engine operations include `fsync` to disk, which dominates latency. Adding a deadline with persistence takes 12.5 ms. A full open-query-save round-trip on 1,000 entities completes in 8.3 ms.

4.3 File Size and I/O Performance

Table 3 reports file size and persistence performance. The binary format achieves 5–6× compression over equivalent JSON representation.

The file header occupies 92 bytes. Individual entity block sizes vary by type: Deadline blocks average 311 bytes, Schedule blocks 363 bytes, Decay blocks 291 bytes, and Duration blocks 246 bytes. The per-entity overhead (21 bytes for type tag, UUID, and length prefix) is modest relative to JSON payloads.

Save latency includes full entity serialization, atomic write (temp file + rename), and `fsync` to ensure durability. Open latency includes header parsing and entity deserialization. At 10,000 entities the save/open cycle completes in 74 ms total. At 1,000 entities (a typical agent workload), the cycle is under 8 ms.

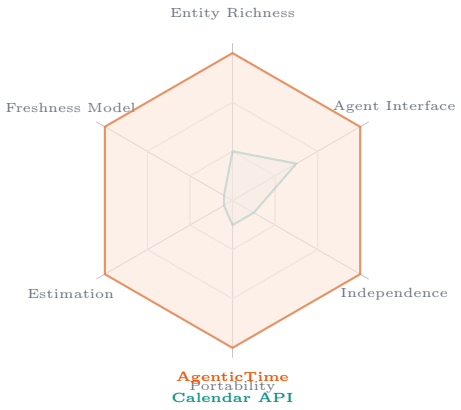

Figure 7: Radar chart comparing AgenticTime (orange) with Calendar APIs (teal) across six dimensions. Each axis ranges from 0 (center) to 3 (outer ring). AgenticTime scores highest across all dimensions due to its typed entities, mathematical models, and agent-native interface.

4.4 Comparison with Existing Systems

Table 4 compares AgenticTime's capabilities with existing temporal systems across eight dimensions.

4.5 Capacity Projections

Table 5 projects real-world storage requirements based on typical usage patterns. At the measured binary format efficiency, a single `.atime` file can support years of continuous agent scheduling within modest storage budgets.

The projections use the measured average entity size of 402 bytes from the mixed-type benchmark (Table 3). Even the most aggressive enterprise scenario—200 new temporal entities per day—produces under 30 MB per year, well within a single file's practical limits. A personal assistant creating 5 entities per day would not exceed 1 MB per year.

5 Discussion

Decay as a first-class temporal primitive. The decision to model information freshness as a core entity type—rather than a metadata annotation—reflects a fundamental observation: for AI agents, *when* information was created is as important as *what* the information says. The Ebbinghaus-inspired decay curves [8, 24] provide a mathematically rigorous model for this intuition, enabling proactive staleness detection rather than reactive discovery.

PERT estimation for agents. Duration estimation with confidence intervals is a well-studied problem in project management [14, 20], but it has not been applied to AI agent scheduling. By exposing PERT as a first-class MCP tool, AgenticTime enables agents to reason about estimation uncertainty, calibrate against historical actuals, and communicate confidence to users.

Table 4: Feature comparison: AgenticTime versus existing temporal tools for AI agents.

Feature	**Calendar API**	**Cron**	**Jira/Linear**	**Temporal.io**	**LLM Prompt**	**AgenticTime**
Typed entities	1 (event)	1 (job)	1 (issue)	1 (workflow)	0	**5**
Decay curves	×	×	×	×	×	✓
PERT estimation	×	×	×	×	×	✓
Conflict detection	Basic	×	×	×	×	✓
Sequence deps	×	×	Partial	✓	×	✓
Integrity checks	×	×	×	×	×	BLAKE3
Portable artifact	×	×	×	×	×	`.atime`
MCP interface	×	×	×	×	×	19 tools
Zero dependencies	×	✓	×	×	✓	✓

Table 5: Real-world capacity projections for `.atime` files based on measured per-entity size of 402 bytes (mixed type average).

Use Case	**Ent./Day**	**MB/Year**	**Years/GB**
Personal assistant	5	0.7	1,400+
Dev copilot	15	2.2	450+
Team scheduler	50	7.3	136+
Enterprise agent	200	29.3	34+

Temporal debt as a scheduling primitive. Cunningham's technical debt metaphor [5] is widely used but rarely quantified. AgenticTime's Decay entity can model deferred work as compounding debt: the "half-life" becomes an interest rate, and the current decay value represents the accumulated cost. Category-specific rates (exponential for security, linear for documentation) formalize what engineering teams know intuitively.

What this enables. AgenticTime makes several previously impossible capabilities practical: (1) mathematical information freshness modeling with proactive alerts, (2) PERT-based duration estimation with historical calibration, (3) automatic schedule conflict detection across agent workflows, (4) dependency-constrained sequence execution with blocking semantics, and (5) portable temporal state that persists across sessions, machines, and LLM providers.

Limitations. The current `.atime` format does not support concurrent writers; file-level locking is required for multi-agent scenarios. Schedule conflict detection is $O(n^2)$ pairwise comparison (56 μs at 100 schedules), which may require interval tree optimization for workloads exceeding 1,000 concurrent schedules. File save rewrites all entities (59.8 ms at 10K), which could be optimized with incremental writes. The format stores temporal entities without causal relationships between them; integration with AgenticMemory [17] is needed for full temporal-causal reasoning. BLAKE3 checksums verify integrity but do not provide encryption; sensitive temporal data requires additional protection layers.

Future work. Planned extensions include: (1) timeline fork simulation for what-if analysis (Track T2), (2) deadline prophecy using Monte Carlo simulation over PERT distributions (Track T3), (3) temporal debt with compounding interest rates by category (Track T4), (4) chronogravity priority warping as deadlines approach (Track T4), (5) temporal anomaly detection for schedule pattern violations (Track T5), and (6) temporal wormholes for cross-project timeline linking (Track T6).

Sister integration. AgenticTime is designed as part of the Agentra ecosystem. Decay curves feed into AgenticMemory's confidence scoring (stale facts receive confidence penalties). Schedule entities trigger AgenticVision captures at periodic intervals. Sequence steps map to AgenticCodebase code units for deployment pipeline modeling. Temporal operations are signed with AgenticIdentity receipts for audit trails.

6 Conclusion

We have presented AgenticTime, a binary temporal format that gives AI agents structured temporal reasoning: deadlines, durations, schedules, sequences, and decay curves, all persisted in a portable `.atime` file and exposed through 19 MCP tools. The format requires zero external dependencies and achieves sub-microsecond latency for mathematical operations (decay evaluation in 6.3 ns, PERT estimation in 1.8 ns), microsecond-scale entity insertion (1.8 μs), and millisecond-scale persistence (6.0 ms for 1,000 entities).

By modeling temporal state as typed entities with domain-specific mathematical models—PERT three-point estimation with confidence intervals, Ebbinghaus-inspired decay curves with three configurable models, Allen-based conflict detection, and dependency-constrained sequence execution—AgenticTime enables capabilities that no existing calendar API, project management tool, or scheduling system provides.

The system is implemented in Rust across 4 crates (core, MCP server, CLI, FFI), validated by 42 tests, and available as open source under dual MIT/Apache-2.0 license. Cross-language bindings for Python (PyO3), JavaScript (WebAssembly), and C (FFI) ensure broad integration. AgenticTime is the first binary format purpose-built for AI agent temporal reasoning, portable across any LLM,

and deployable with zero infrastructure.

References

[1] James F Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983.

[2] John R Anderson, Daniel Bothell, Michael D Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

[3] Anthropic. Model context protocol specification. `https://spec.modelcontextprotocol.io/`, 2024. Accessed 2025.

[4] Atlassian. Jira: Issue and project tracking software. `https://www.atlassian.com/software/jira`, 2024. Accessed 2025.

[5] Ward Cunningham. The WyCash portfolio management system. In *ACM SIGPLAN OOPS Messenger*, volume 4, pages 29–30. ACM, 1992.

[6] Rina Dechter, Itay Meiri, and Judea Pearl. Temporal constraint networks. *Artificial Intelligence*, 49(1–3):61–95, 1991.

[7] Bernard Desruisseaux. Internet calendaring and scheduling core object specification (iCalendar), 2009.

[8] Hermann Ebbinghaus. Über das gedächtnis. 1885. English translation: Memory: A Contribution to Experimental Psychology, 1913.

[9] Google LLC. Google Calendar API. `https://developers.google.com/calendar/api`, 2024. Accessed 2025.

[10] Brook Heisler and Jorge Aparicio. Criterion.rs: Statistics-driven micro-benchmarking in Rust. `https://github.com/bheisler/criterion.rs`, 2023. Accessed 2025.

[11] IEEE and The Open Group. IEEE std 1003.1-2017: crontab. `https://pubs.opengroup.org/onlinepubs/9699919799/utilities/crontab.html`, 2017. POSIX.1-2017.

[12] Philippe Kruchten, Robert L Nord, and Ipek Ozkaya. Technical debt: From metaphor to theory and practice. *IEEE Software*, 29(6):18–21, 2012.

[13] Linear Inc. Linear: Modern issue tracking. `https://linear.app/`, 2024. Accessed 2025.

[14] D G Malcolm, J H Roseboom, C E Clark, and W Fazar. Application of a technique for research and development program evaluation. In *Operations Research*, volume 7, pages 646–669. INFORMS, 1959.

[15] Nicola Muscettola. HSTS: Integrating planning and scheduling. In *Intelligent Scheduling*, pages 169–212. Morgan Kaufmann, 1994.

[16] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. `https://github.com/BLAKE3-team/BLAKE3`, 2020. Accessed 2025.

[17] Omoshola Owolabi. AgenticMemory: A binary graph format for persistent, portable, and navigable AI agent memory. 2026. Available at `https://github.com/agentralabs/agentic-memory/tree/main/paper`.

[18] Charles Packer, Joseph E Gonzalez, Dawn Song, Raymond J Mooney, et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[19] Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.

[20] Michael L Pinedo. Scheduling: Theory, algorithms, and systems. 2016.

[21] David E Smith, Jeremy Frank, and Ari K Jónsson. Bridging the gap between planning and scheduling. *The Knowledge Engineering Review*, 15(1):47–83, 2000.

[22] Temporal Technologies Inc. Temporal: Durable execution platform. `https://temporal.io/`, 2024. Accessed 2025.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[24] John T Wixted. The psychology and neuroscience of forgetting. *Annual Review of Psychology*, 55:235–269, 2004.